Networks Assignment - 02 Lab Report

Application: Client-Server programming using both TCP and UDP sockets (1st application)

Introduction

In this lab, we implemented a client-server communication system using both TCP and UDP sockets.

- TCP was used for connection establishment and negotiation of a UDP port (reliable, connection-oriented).
- UDP was used for actual data transfer (faster, lightweight, connectionless).

The goal was to design a two-stage protocol, implement a concurrent server with scheduling policies (FCFS and Round-Robin), and measure throughput for TCP and UDP.

Background Theory

TCP (Transmission Control Protocol)

TCP is a reliable, connection-oriented protocol that ensures the accurate and orderly delivery of data between a client and a server. It breaks data into segments, numbers them, and reassembles them correctly at the destination. It's the foundation for many internet applications like web browsing.

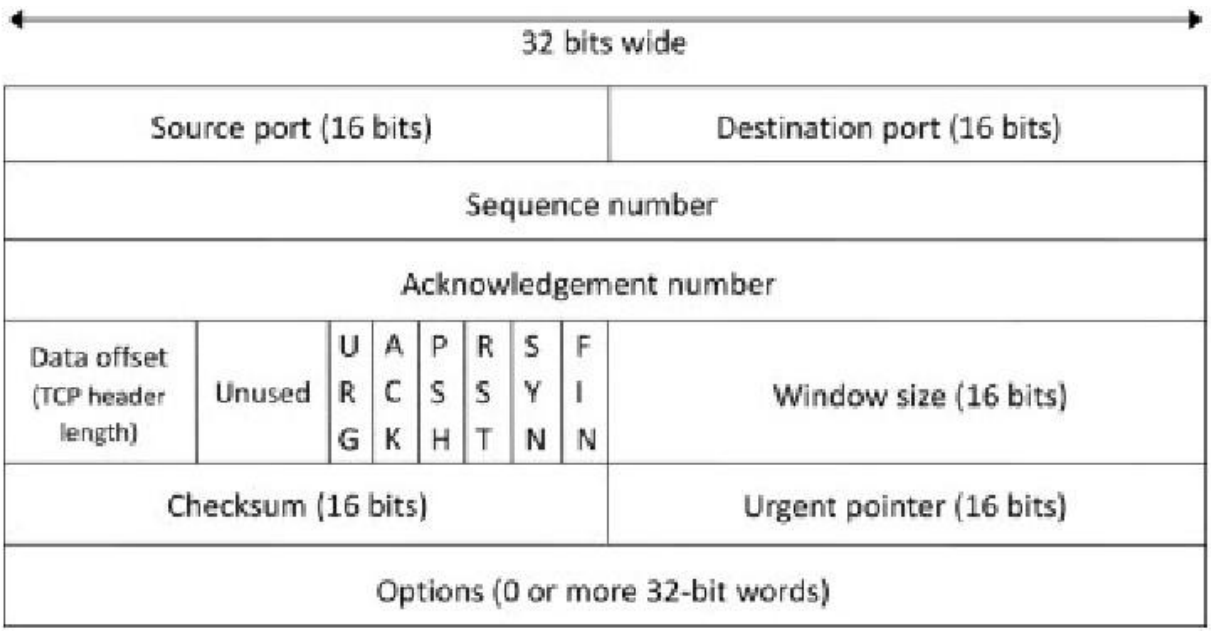
Key Fields in a TCP Header:

- Source and Destination Ports: These 16-bit fields identify the specific applications on the sending and receiving devices, respectively.
- Sequence Number: A 32-bit field that helps order segments and reassemble data correctly.
- Acknowledgement Number: A 32-bit field used by the receiver to confirm receipt of data and request the next segment.
- Control Flags: Six 1-bit flags that control the connection state (e.g., SYN to initiate a connection, FIN to terminate it).
- Window Size: A 16-bit field used for flow control, preventing the sender from overwhelming the receiver.

How it Works: Three-Way Handshake

Before data transfer, TCP establishes a connection using a three-way handshake to ensure both sides are ready.

- SYN: The client sends a segment to the server to request a connection.
- SYN-ACK: The server responds, acknowledging the client's request and sending its own request.
- ACK: The client sends a final acknowledgement, and the connection is established.



User Datagram Protocol (UDP):

UDP is a simple, connectionless protocol used for time-sensitive applications where speed is more important than perfect reliability, such as streaming video, online gaming, and voice over IP (VoIP). Unlike TCP, UDP doesn't establish a connection or guarantee delivery.

Key Fields in a UDP Header:

- Source Port: A 16-bit field that identifies the port number of the originating application on the sender's device.
- Destination Port: A 16-bit field that identifies the port number of the receiving application on the destination device.
- Length: This 16-bit field specifies the entire length of the UDP datagram, including both the header and the user data.
- Checksum: A 16-bit field used for error checking. It allows the receiver to verify the integrity of the data and header. The checksum calculation is optional for IPv4 but mandatory for IPv6.

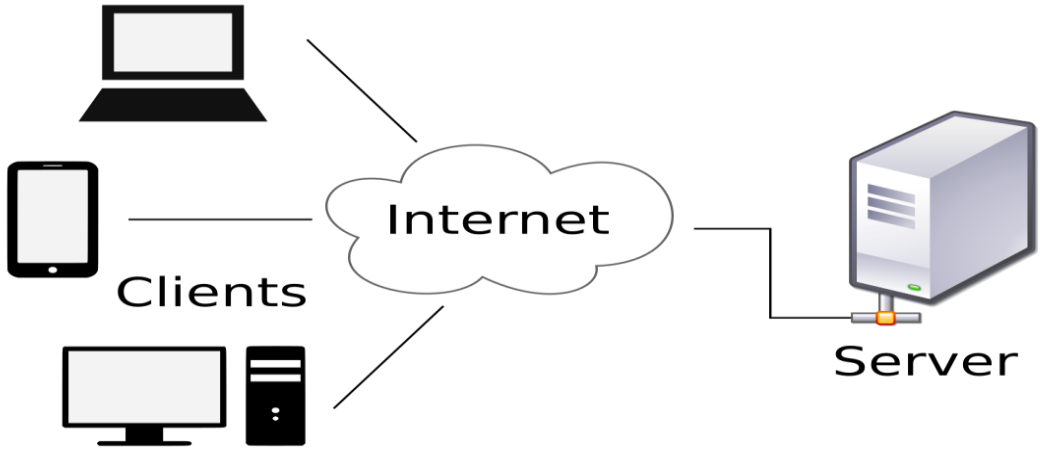
How it Works

UDP is often called a "fire-and-forget" protocol because it sends data without first establishing a connection. The sender simply transmits a datagram and doesn't wait for any acknowledgment of receipt. This makes it much faster and more efficient than TCP but also less reliable. If a datagram is lost or arrives out of order, UDP does not attempt to resend it or reorder the packets. This behavior is ideal for applications where a small amount of data loss is acceptable, as long as the data arrives quickly.

Bit	+0..7	+8..15	+16..23	+24..31
0	Source Port		Destination Port	
32	Length		Checksum	
...	*Payload*			

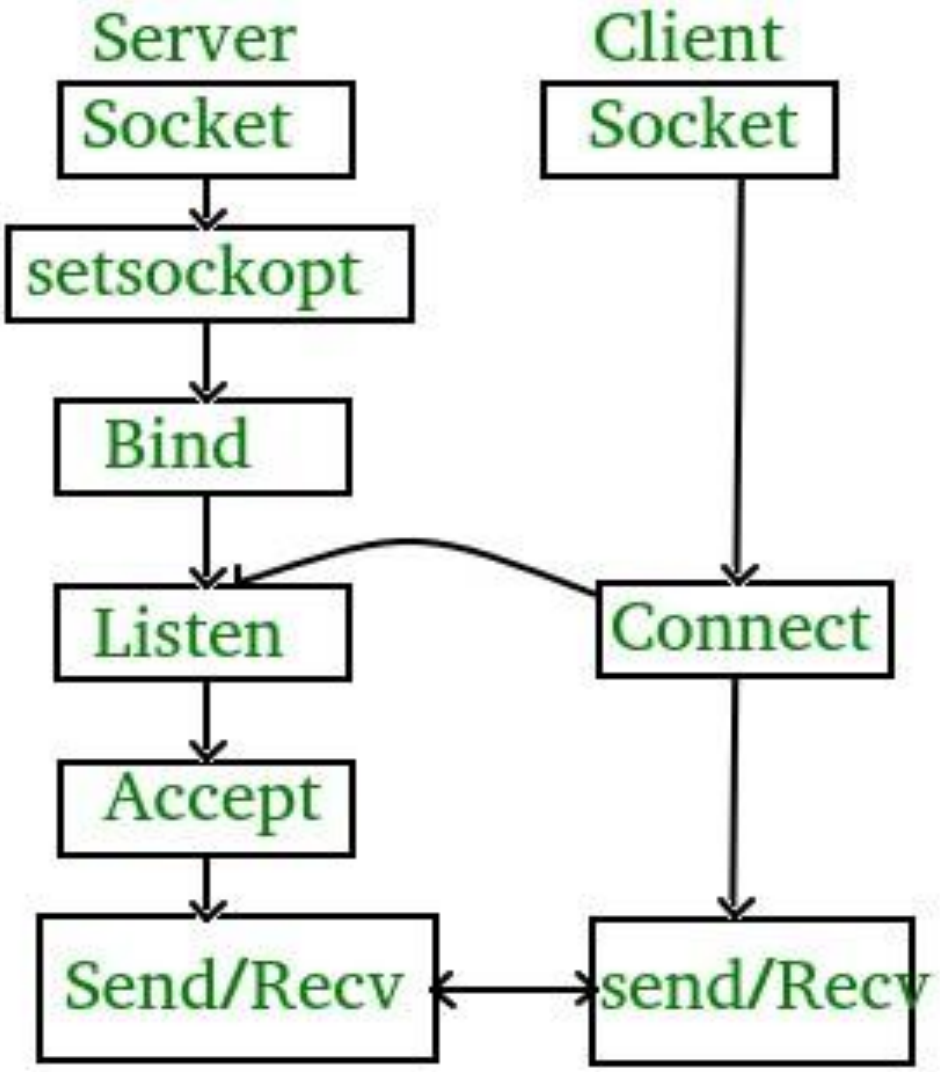
Client-Server Model

- Client initiates a connection.
- The server listens on a port and responds to requests.
- We extended the server to handle multiple clients concurrently.



Concurrent Servers and Scheduling

- Concurrent server: serves multiple clients at the same time.
- Implemented using multi-threading and a request queue.
- FCFS (First Come First Serve): clients are served in the order they arrive.
- Round-Robin (RR): clients are served in time slices, cycling fairly among them.



Implementation

Client.c

- Connects to the server using TCP.

```c
if ((tcp_socket = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
    perror("TCP socket failed");
    exit(EXIT_FAILURE);
}

memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(DEFAULT_TCP_PORT);

if (inet_pton(AF_INET, server_ip, &server_addr.sin_addr) <= 0) {
    perror("Invalid address");
    exit(EXIT_FAILURE);
}

if (connect(tcp_socket, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
    perror("TCP connection failed");
    exit(EXIT_FAILURE);
}

printf("Connected to server %s via TCP\n", server_ip);
```

- Sends Type-1 (Request) message.

```c
char *request = "1:18:Request for UDP port";
send(tcp_socket, request, strlen(request), 0);
printf("Sent Type 1 message\n");
```

- Receives Type-2 (Response) with UDP port.

```c
int n = read(tcp_socket, buffer, BUFFER_SIZE);
if (n > 0) {
    buffer[n] = '\0';
    printf("Received Type 2 message: %s\n", buffer);

    if (sscanf(buffer, "2:4:%d", &udp_port) == 1) {
        printf("Negotiated UDP port: %d\n", udp_port);
    } else {
        printf("Error parsing UDP port\n");
        close(tcp_socket);
        exit(EXIT_FAILURE);
    }
}
close(tcp_socket);
```

- Closes TCP.
- Switches to UDP → sends Type-3 (Data) messages (from sample.txt).

```c
if ((udp_socket = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
    perror("UDP socket failed");
    exit(EXIT_FAILURE);
}

struct timeval tv;
tv.tv_sec = 5;
tv.tv_usec = 0;
setsockopt(udp_socket, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

server_addr.sin_port = htons(udp_port);

FILE *fp = fopen("sample.txt", "r");
if (!fp) {
    perror("File open failed");
    exit(EXIT_FAILURE);
}

char response[BUFFER_SIZE];
char buffer1[BUFFER_SIZE - 10];
total_msgs = 0;
total_bytes_sent = 0;
```

```c
start_time = get_time_sec();

// Send the file multiple times based on iterations
for (int round = 1; round <= iterations; round++) {
    rewind(fp); // Reset file pointer for each iteration

    while (fgets(buffer1, sizeof(buffer1), fp)) {
        buffer1[strcspn(buffer1, "\n")] = '\0';
        int datalen = strlen(buffer1);
        snprintf(response, BUFFER_SIZE, "3:%d:%s", datalen, buffer1);
        sendto(udp_socket, response, strlen(response), 0,(struct sockaddr *)&server_addr, sizeof(server_addr));
        total_msgs++;
        total_bytes_sent += datalen;
    }
}

// Send final Bye message
sendto(udp_socket, "Bye", strlen("Bye"), 0, (struct sockaddr *)&server_addr, sizeof(server_addr));
total_msgs++;

end_time = get_time_sec();
total_time = end_time - start_time;
```

The client sends the file multiple times in small chunks (line by line) for each iteration. This simulates repeated file transfers and provides consistent throughput measurements. In TCP, the same logic applies, except data is sent over a persistent connection instead of attaching the destination each time.

- Sends “Bye” to the end session.
- This is a special type of message that indicates server that the client has completed sending the data.
- Receives Type-4 (Acknowledgment).

```c
socklen_t server_len = sizeof(server_addr);
n = recvfrom(udp_socket, buffer, BUFFER_SIZE, 0, (struct sockaddr *)&server_addr, &server_len);
if (n > 0) {
    buffer[n] = '\0';
    printf("Received Type 4 response: %s\n", buffer);
}
else {
    printf("Server did not respond to Bye (possible packet loss)\n");
}

close(udp_socket);
fclose(fp);
printf("UDP communication completed\n");
```

- Measures throughput and writes results into results.csv (UDP) and results2.csv (TCP).

Server.c

Starts listening on a TCP port.

```c
if ((server_tcp_socket = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
    perror("TCP socket failed");
    exit(EXIT_FAILURE);
}

if (setsockopt(server_tcp_socket, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt))) {
    perror("setsockopt failed");
    exit(EXIT_FAILURE);
}

address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(port);

if (bind(server_tcp_socket, (s                                    zeof(address)) < 0) {
    perror("TCP bind failed");
    exit(EXIT_FAILURE);
}

if (listen(server_tcp_socket, MAX_CLIENTS) < 0) {
    perror("TCP listen failed");
    exit(EXIT_FAILURE);
}

printf("Server started on port %d with %s scheduling\n", port, policy == FCFS ? "FCFS" : "Round-Robin");
```

```
#define MAX_CLIENTS 10
Maximum parallel TCP Connections
Expands to:
10
```

```c
pthread_t threads[4];
for (int i = 0; i < 4; i++) {
    pthread_create(&threads[i], NULL, thread, NULL);
}


while (1) {

    int new_socket;
    struct sockaddr_in client_addr;
    socklen_t client_len = sizeof(client_addr);

    if ((new_socket = accept(server_tcp_socket, (struct sockaddr *)&client_addr, &client_len)) < 0) {
        perror("TCP accept failed");
        continue;
    }

    printf("New client connected: %s:%d\n", inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));


    ClientRequest req;
    req.client_socket = new_socket;
    req.client_addr = client_addr;
    req.served = 0;
    req.successfull_recieved =0;

    char buffer[BUFFER_SIZE] = {0};
    int udp_port;
```

On new client connection:

- Receives Type-1 Request.

```c
// Read Type 1 message
read(new_socket, buffer, BUFFER_SIZE);
printf("Client: %s\n", buffer);


udp_port = 9000 + (rand() % 1000);
req.udp_port = udp_port;
```

- Sends Type-2 Response with randomly generated UDP port.

```c
char response[BUFFER_SIZE];
snprintf(response, BUFFER_SIZE, "2:4:%d", udp_port);
send(new_socket, response, strlen(response), 0);
printf("Sent Type 2 message with UDP port: %d\n", udp_port);
```

–> Queues request in the RequestQueue.

RequestQueue is implemented in 2 ways namely:

First-Come, First-Serve (FCFS) Scheduling

- Definition: Clients are served strictly in the order they arrive (queue-based).
- Mechanism:
    - Each incoming client request is added to a request queue.
    - The server assigns a thread to handle the first request in the queue.
    - Once completed, the next request is served.
- Advantages:
    - Simple to implement.
    - Fair in the sense that no client can overtake another.
- Disadvantages:
    - Long requests block shorter ones (convoy effect).
    - Not suitable for highly interactive workloads.

```
void enqueue(ClientRequest req) {
    pthread_mutex_lock(&queue.lock);

    if (queue.count < QUEUE_SIZE) {
        queue.rear = (queue.rear + 1) % QUEUE_SIZE;
        queue.requests[queue.rear] = req;
        queue.count++;
        pthread_cond_signal(&queue.not_empty);
    }

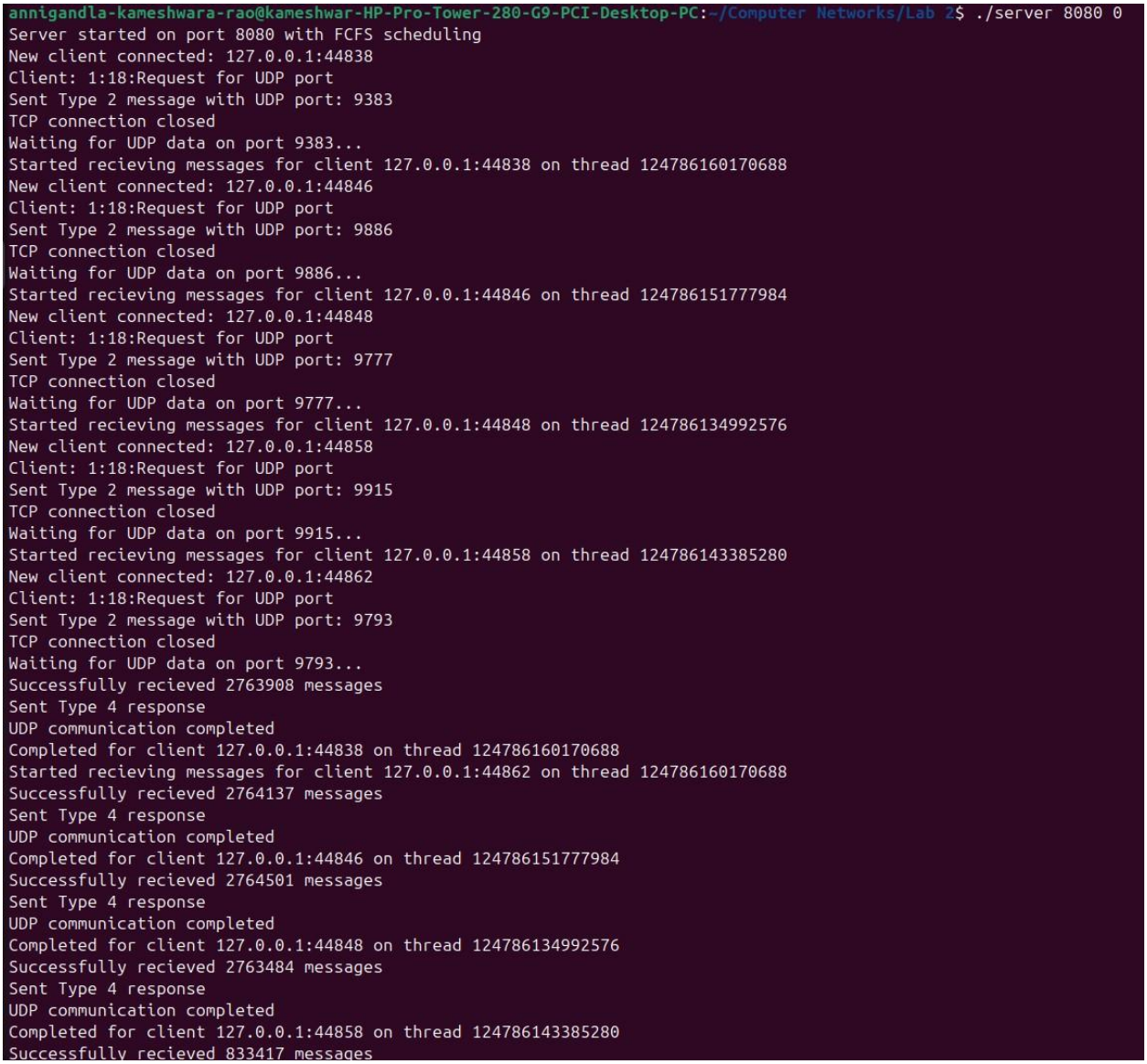
    pthread_mutex_unlock(&queue.lock);
}

ClientRequest* dequeue_fcfs() {
    pthread_mutex_lock(&queue.lock);

    while (queue.count == 0) {
        pthread_cond_wait(&queue.not_empty, &queue.lock);
    }

    ClientRequest *req = &queue.requests[queue.front];
    queue.front = (queue.front + 1) % QUEUE_SIZE;
    queue.count--;

    pthread_mutex_unlock(&queue.lock);
    return req;
}
```



- 5 clients connected sequentially to the server (4 threaded), so the requested udp ports are sent to them sequentially.
- However, the first 4 were running parallelly (on 4 threads) and the 4th client is blocked (since 4 threads are busy), also you can see the sequence of the started receiving messages (1->2->3->4).
- Now we can observe that after the 1st client messages were processed and it gets completed now the thread 1 starts receiving from the 5th client (First Come First Serve).
- So finally the order of operations in the server:
    1. Client 1 connects.
    2. Starts recieving Client 1 messages on thread 1.

3. Client 2 connects.
4. Starts recieving Client 2 messages on thread 2.
5. Client 3 connects.
6. Starts recieving Client 3 messages on thread 3.
7. Client 4 connects.
8. Starts recieving Client 4 messages on thread 4.
9. Client 5 connects
10. Thread 1 completes receiving Client 1 messages.
11. Starts recieving Client 5 messages on thread 1.
12. Thread 2 completes receiving Client 2 messages. 13. Thread 3 completes receiving Client 3 messages. 14. Thread 4 completes receiving Client 4 messages. 15. Thread 1 completes receiving Client 5 messages.

Round-Robin (RR) Scheduling

- Definition: Clients are served in fixed time slices, cycling through them repeatedly.
- Mechanism:
  - Each client gets a time quantum (e.g., 10000 messages).
  - If the client's request is not finished, it is placed back in the queue.
  - The next client in the queue is served.
- Advantages:
  - Ensures fairness — every client gets CPU/network attention. ○ Prevents starvation (no request is ignored indefinitely).
- Disadvantages:
  - Context switching overhead.
  - If the quantum is too small → too much overhead.
  - If the quantum is too large → behaves like FCFS.

```c
ClientRequest* dequeue_rr() {
    pthread_mutex_lock(&queue.lock);

    while (queue.count == 0) {
        pthread_cond_wait(&queue.not_empty, &queue.lock);
    }

    // Round Robin: Find the next client that hasn't been served yet
    ClientRequest *req;
    int found = 0;

    for (int i = 0; i < queue.count; i++) {
        int index = (queue.front + i) % QUEUE_SIZE;
        if (!queue.requests[index].served) {
            req = &queue.requests[index];
            queue.requests[index].served = 1; // Mark as served
            found = 1;
            break;
        }
    }

    // If all clients have been served in this cycle, reset and start new cycle
    if (!found) {
        for (int i = 0; i < queue.count; i++) {
            int index = (queue.front + i) % QUEUE_SIZE;
            queue.requests[index].served = 0; // Reset all
        }
        req = &queue.requests[queue.front];
        queue.requests[queue.front].served = 1;
    }

    pthread_mutex_unlock(&queue.lock);
    return req;
}
```

```c
int noofmsgs=0;
while(noofmsgs<TIME_SLOT){
    int n = recvfrom(udp_socket, buffer, BUFFER_SIZE, 0, (struct sockaddr *)&client_addr, &client_len);
    if (n > 0) {
        buffer[n] = '\0';
        noofmsgs++;
        req->successfull_recieved++;
        //printf("Client: %s\n", buffer);
    }
    if (n < 0) {
        perror("Timedout or error");
        printf("Successfully recieved %d messages\n", req->successfull_recieved);
        remove_completed(*req);
        close(udp_socket);
        return ;
    }
    if(strcmp("Bye",buffer)==0) {
        // Send Type 4 response
        char *response = "4:26:Data received successfully";
        sendto(udp_socket, response, strlen(response), 0,(struct sockaddr *)&client_addr, client_len);
        printf("Successfully recieved %d messages\n", req->successfull_recieved);
        printf("Sent Type 4 response\n");
        remove_completed(*req);
        close(udp_socket);
        printf("UDP communication completed\n");
        return ;
    }
}

if (policy == RR) {
        printf("Completed time slice for client %s:%d on thread %ld\n", inet_ntoa(req->client_addr.sin_addr), nto
}

return;
```

```c
void remove_completed(ClientRequest req){
    pthread_mutex_lock(&queue.lock);

    for (int i = 0; i < queue.count; i++) {
        int index = (queue.front + i) % QUEUE_SIZE;
        if (queue.requests[index].udp_port == req.udp_port) {
            for (int j = i; j < queue.count - 1; j++) {
                int current_pos = (queue.front + j) % QUEUE_SIZE;
                int next_pos = (queue.front + j + 1) % QUEUE_SIZE;
                queue.requests[current_pos] = queue.requests[next_pos];
            }
            queue.count--;
            queue.rear = (queue.rear - 1 + QUEUE_SIZE) % QUEUE_SIZE;
        }
    }

    if (queue.count > 0) {
        printf("Remaining Connections: ");
        int n = 0;

        for (int i = queue.front; n < queue.count; i = (i + 1) % QUEUE_SIZE) {
            printf("%s:%d, ",inet_ntoa(queue.requests[i].client_addr.sin_addr),ntohs(queue.requests[i].client_addr.sin_port));
            n++;
        }
        printf("\n");
    }
    else {
        printf("No remaining active connections.\n");
    }

    pthread_mutex_unlock(&queue.lock);
}
```

```c
// Send the file multiple times based on iterations
for (int round = 1; round <= iterations; round++) {
    rewind(fp); // Reset file pointer for each iteration

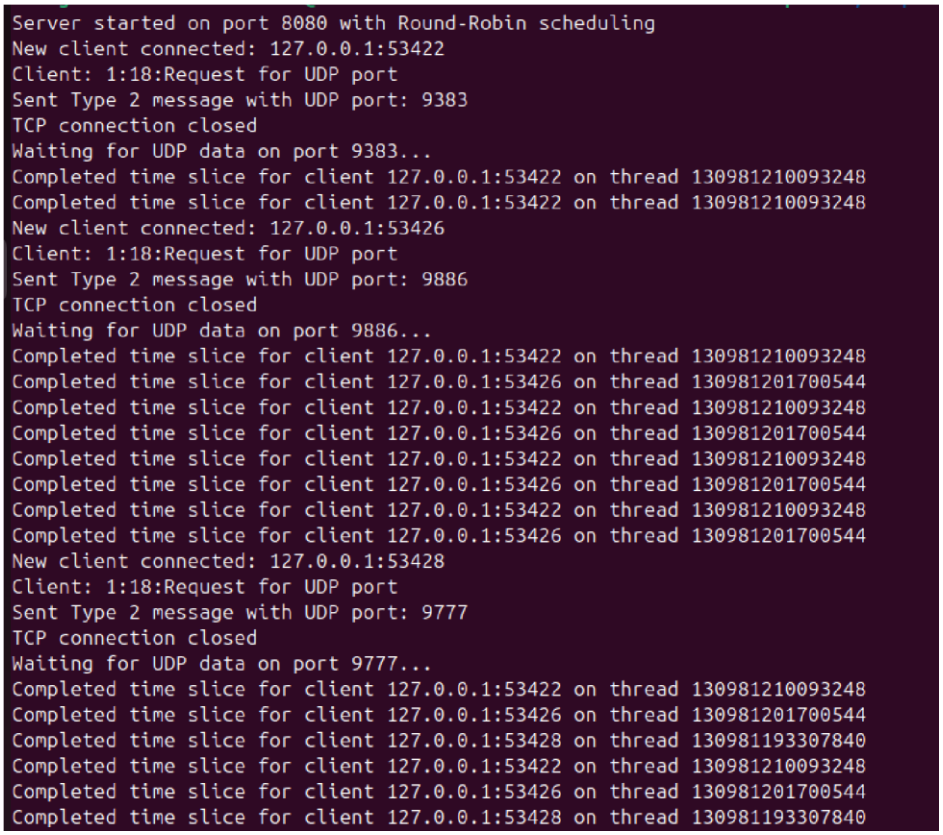
    while (fgets(buffer1, sizeof(buffer1), fp)) {
        buffer1[strcspn(buffer1, "\n")] = '\0';
        int datalen = strlen(buffer1);
        snprintf(response, BUFFER_SIZE, "3:%d:%s", datalen, buffer1);
        sendto(udp_socket, response, strlen(response), 0,(struct sockaddr *)&server_addr, sizeof(server_addr));
        total_msgs++;
        total_bytes_sent += datalen;
    }
    //printf("sent %d round\n",round);
}
```

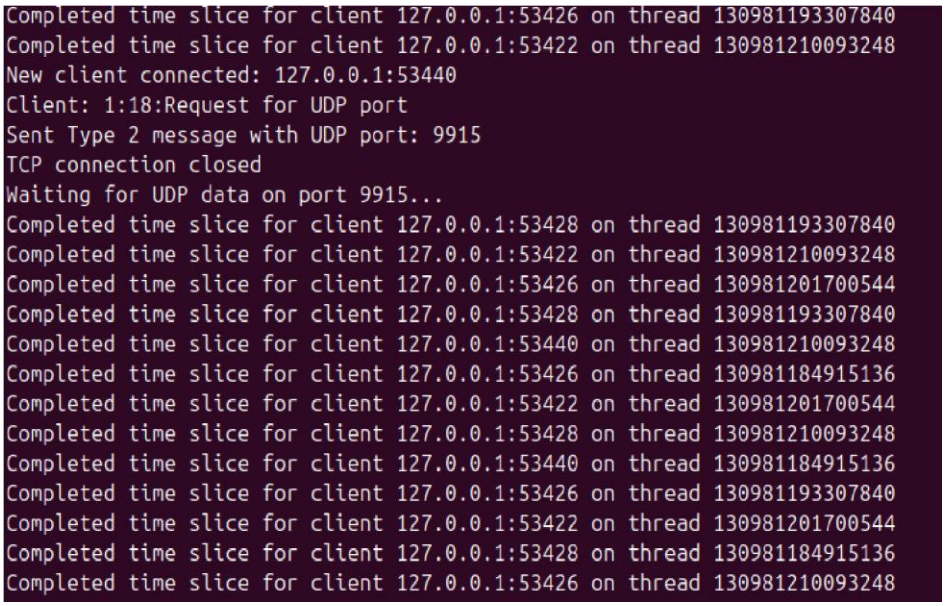
Function: remove_completed(ClientRequest req)

1. **Thread-Safe Removal**
    - Uses pthread_mutex_lock to ensure safe access when removing a completed client request from the circular queue.
2. **Search & Delete**
    - Finds the request by matching udp_port and removes it by **shifting subsequent requests** to maintain queue order.
3. **Queue Update**
    - Decrements queue.count and updates queue.rear using modulo arithmetic for circular buffer management.
4. **Display Active Clients**

- Prints the list of remaining active connections or shows **“No remaining active connections”** if the queue is empty.

5. **Efficient Resource Management**
   - Keeps the queue clean and ready for new incoming client requests.





- 4 clients connected sequentially to the server (4 threaded), so the requested udp ports are sent to them sequentially.

- All the 4 were running parallelly (on 4 threads), also you can see the sequence of the clients completing time slicing (Eg: 1->2->3->1->4->2->3->1->2->3->4).

```
Completed time slice for client 127.0.0.1:53428 on thread 130981193307840
Completed time slice for client 127.0.0.1:53426 on thread 130981184915136
Completed time slice for client 127.0.0.1:53440 on thread 130981210093248
Successfully recieved 1307574 messages
Sent Type 4 response
Remaining Connections: 127.0.0.1:53426, 127.0.0.1:53428, 127.0.0.1:53440,
UDP communication completed
Completed time slice for client 127.0.0.1:53440 on thread 130981184915136
Completed time slice for client 127.0.0.1:53440 on thread 130981210093248
Completed time slice for client 127.0.0.1:53428 on thread 130981184915136
Completed time slice for client 127.0.0.1:53428 on thread 130981193307840
Completed time slice for client 127.0.0.1:53440 on thread 130981210093248
Completed time slice for client 127.0.0.1:53426 on thread 130981201700544
Completed time slice for client 127.0.0.1:53428 on thread 130981193307840
Completed time slice for client 127.0.0.1:53440 on thread 130981210093248
Completed time slice for client 127.0.0.1:53428 on thread 130981193307840
Completed time slice for client 127.0.0.1:53440 on thread 130981210093248
Successfully recieved 1293094 messages
Sent Type 4 response
Remaining Connections: 127.0.0.1:53428, 127.0.0.1:53440,
UDP communication completed
Completed time slice for client 127.0.0.1:53440 on thread 130981210093248
Completed time slice for client 127.0.0.1:53440 on thread 130981193307840
Completed time slice for client 127.0.0.1:53428 on thread 130981210093248
Completed time slice for client 127.0.0.1:53440 on thread 130981193307840
Successfully recieved 1226014 messages
Sent Type 4 response
Remaining Connections: 127.0.0.1:53440,
UDP communication completed
Completed time slice for client 127.0.0.1:53440 on thread 130981193307840
Completed time slice for client 127.0.0.1:53440 on thread 130981210093248
Completed time slice for client 127.0.0.1:53440 on thread 130981193307840
Completed time slice for client 127.0.0.1:53440 on thread 130981210093248
Completed time slice for client 127.0.0.1:53440 on thread 130981193307840
Successfully recieved 1256821 messages
Sent Type 4 response
No remaining active connections.
UDP communication completed
Timedout or error: Resource temporarily unavailable
Timedout or error: Resource temporarily unavailable
Successfully recieved 1256821 messages
No remaining active connections.
Successfully recieved 1256821 messages
No remaining active connections.
```

- Now we can observe the sequence of completion 1->2->3->4.
- Here even though some of the clients connected late, they were not blocked by the previous requested ones.
- All were given equal time slices, so all the requests progressed (Round Robin).

Handles UDP communication:

- Receives Type-3 Data messages.

```c
int noofmsgs=0;
while(1){
    int n = recvfrom(udp_socket, buffer, BUFFER_SIZE, 0, (struct sockaddr *)&client_addr, &client_len);
    if (n > 0) {
        buffer[n] = '\0';
        noofmsgs++;
        req->successfull_recieved++;
        //printf("Client: %s\n", buffer);
    }
    if (n < 0) {
        perror("Timedout or error");
        printf("Successfully recieved %d messages\n", req->successfull_recieved);
        remove_completed(*req);
        close(udp_socket);
        return ;
    }
```

- Sends Type-4 Response (“Data received successfully”).

```c
if(strcmp("Bye",buffer)==0) {
    // Send Type 4 response
    char *response = "4:26:Data received successfully";
    sendto(udp_socket, response, strlen(response), 0,(struct sockaddr *)&client_addr, client_len);
    printf("Successfully recieved %d messages\n", req->successfull_recieved);
    printf("Sent Type 4 response\n");
    remove_completed(*req);
    close(udp_socket);
    printf("UDP communication completed\n");
    return ;
}
```

Implements multi-threading and scheduling (FCFS or Round-Robin).

```c
pthread_t threads[4];
for (int i = 0; i < 4; i++) {
    pthread_create(&threads[i], NULL, thread, NULL);
}
```

sample.txt

- A text file of size 1024 Bytes(1KB) whose contents are sent by the client via UDP can be sent multiple times (no of iterations) to generate larger data messages.

Lab 2 > sample.txt

```
a TCP connection from
the client. Then, the client will connect t
o the server using
the server's TCP port already known to the
client. After successful connection,
the client sends a Request Message (Type 1 m
essage) to the server via TCP port to
request a UDP port number from the server f
or future communication. After receiving
the Request Message, the server selects a U
DP port number and sends
this port number back to the client as a Resp
onse Message (Type 2 hello gn
Message) over the TCP connection. After th
is negotiation phase, the TCP connection
on both the server and client should be clo
sed gracefully, releasing the socket resource.
onse Message (Type 2 hello
Message) over the TCP connection. After th
is negotiation phase, the TCP connection
on both the server and client should be clo
sed gracefully, releasing the socket resource.
onse Message (Type 2 hello
Message) over the TCP connection. After
is negotiation phase, the TCP connection
on both the server and client should be clo
hello world!! 1024 bytes are sent
```

```c
// Send the file multiple times based on iterations
for (int round = 1; round <= iterations; round++) {
    rewind(fp); // Reset file pointer for each iteration

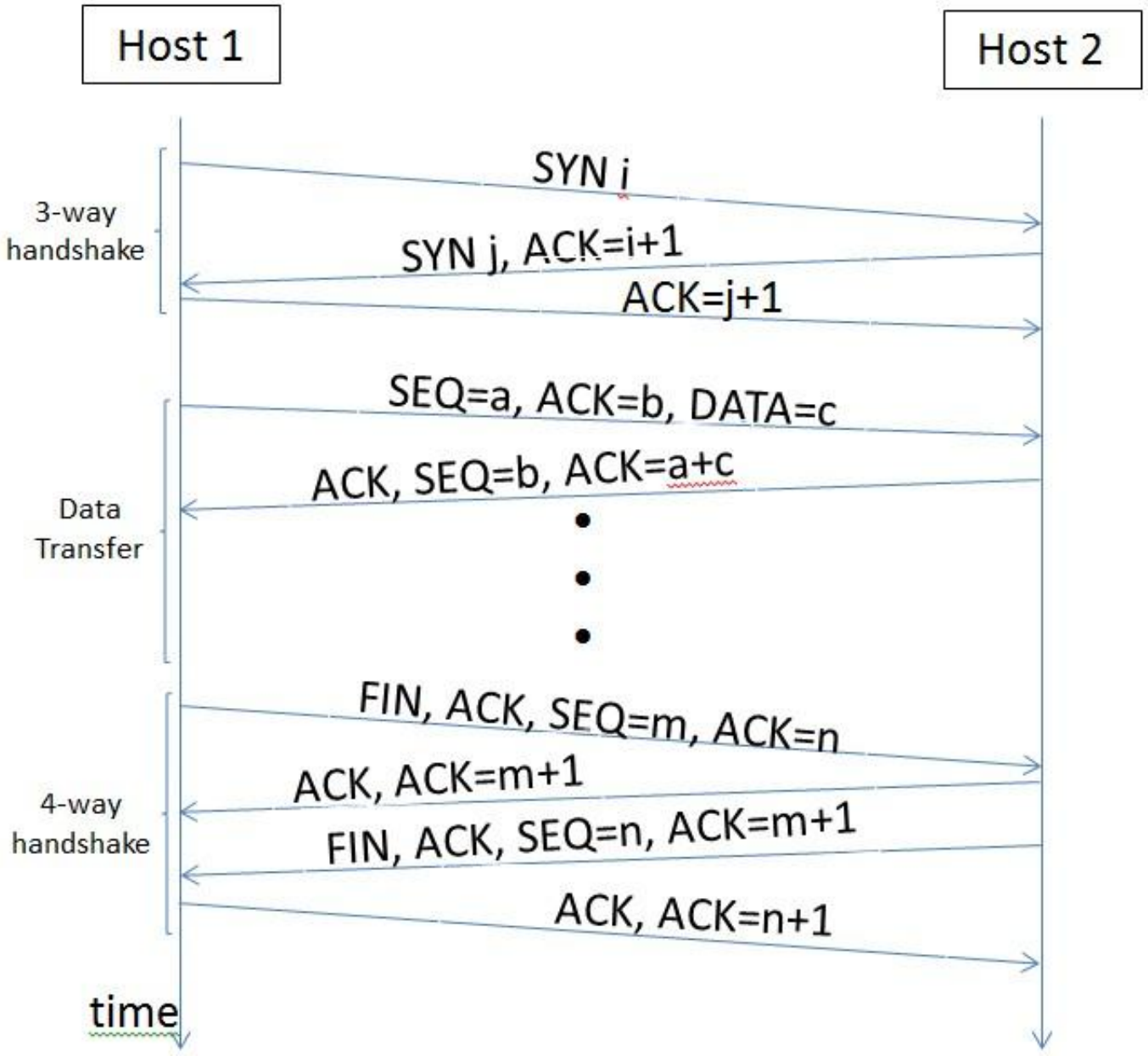
    while (fgets(buffer1, sizeof(buffer1), fp)) {
        buffer1[strcspn(buffer1, "\n")] = '\0';
        int datalen = strlen(buffer1);
        snprintf(response, BUFFER_SIZE, "3:%d:%s", datalen, buffer1);
        sendto(udp_socket, response, strlen(response), 0,(struct sockaddr *)&server_addr, sizeof(server_addr));
        total_msgs++;
        total_bytes_sent += datalen;
    }
    //printf("sent %d round\n",round);
}
```

Protocol Flow

TCP Phase

- Client → Server: Type-1 Request.

○ Server → Client: Type-2 Response (UDP port). ○ TCP connection closed.

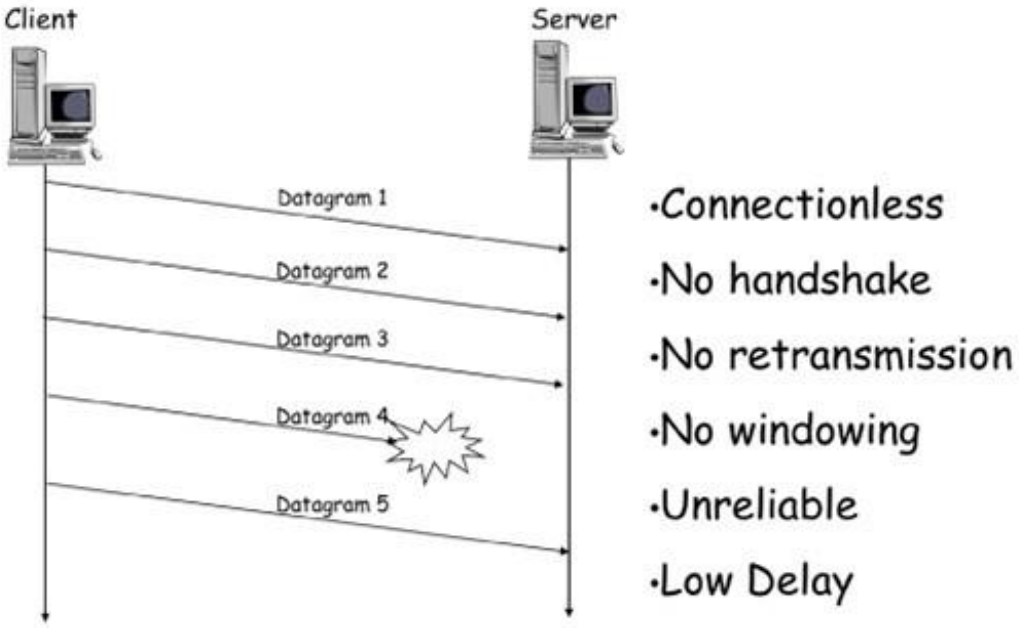


UDP Phase

○ Client → Server: Type-3 Data messages (file contents).

○ Client → Server: “Bye” message.

○ Server → Client: Type-4 Response (acknowledgment). ○ UDP sockets closed.



Results

UDP Results (Done for different scheduling policies & different clients)

- Observed that throughput increases with message size (up to ~225 Mbps).
- At very small message sizes, throughput is low due to overhead per packet.

The experiment measures UDP throughput for varying message sizes ranging from 1 KB up to 10,240 KB (~10 MB).

Analysis in Client:

```
Connected to server 127.0.0.1 via TCP
Sent Type 1 message
Received Type 2 message: 2:4:9915
Negotiated UDP port: 9915

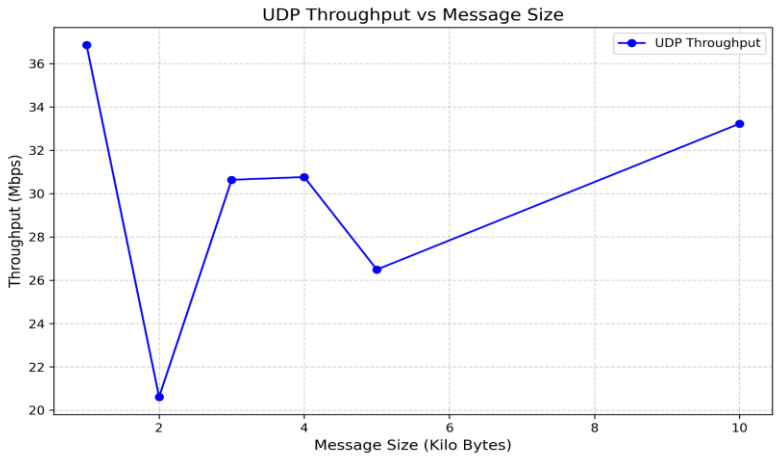
TCP Transmission Summary:
Total Messages Sent : 1
Total Bytes Sent    : 18 bytes
Total Time Taken    : 0.000100 seconds
Throughput          : 1.441 Mbps

TCP connection closed

UDP Transmission Summary:
Total Messages Sent : 1350001
Total Bytes Sent    : 51200000 bytes
Total Time Taken    : 1.839 seconds
Throughput          : 222.740 Mbps

Received Type 4 response: 4:26:Data received successfully
UDP communication completed
```

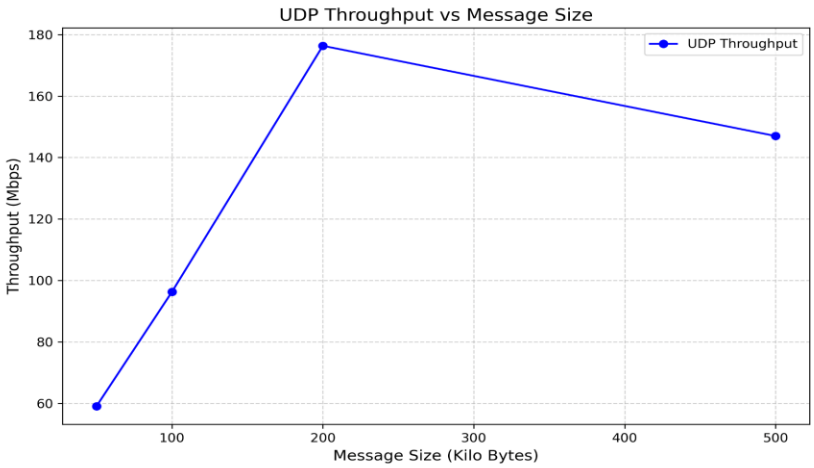
Key Observations from the Plot across different messages sizes



(a) Small Message Sizes (1 KB ⟶ 10 KB)

- Throughput is low and somewhat unstable in this region.
- At 1 KB, throughput ≈ 36 Mbps.

- At 2 KB, throughput ≈ 21 Mbps (unexpected dip).
- From 3 KB ⟶ 10 KB, throughput rises slowly to ≈ 33 Mbps.
- Reason:
    - For smaller messages, protocol overhead (UDP/IP/Ethernet headers) dominates.
    - Context switching and system call overhead reduce overall efficiency.

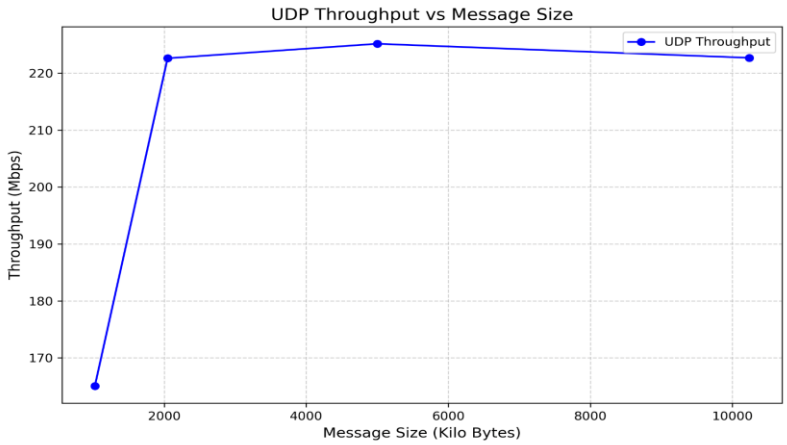


(b) Medium Message Sizes (50 KB ⟶ 500 KB)

- Throughput improves significantly:
    - 50 KB ⟶ ≈ 60 Mbps
    - 100 KB ⟶ ≈ 90 Mbps
    - 200 KB ⟶ ≈ 110 Mbps
    - 500 KB ⟶ ≈ 150 Mbps ●

Reason:

- Larger message sizes reduce per-packet overhead.
- UDP becomes more efficient as fewer packets are required.



(c) Large Message Sizes (1 MB ⟶ 10 MB)

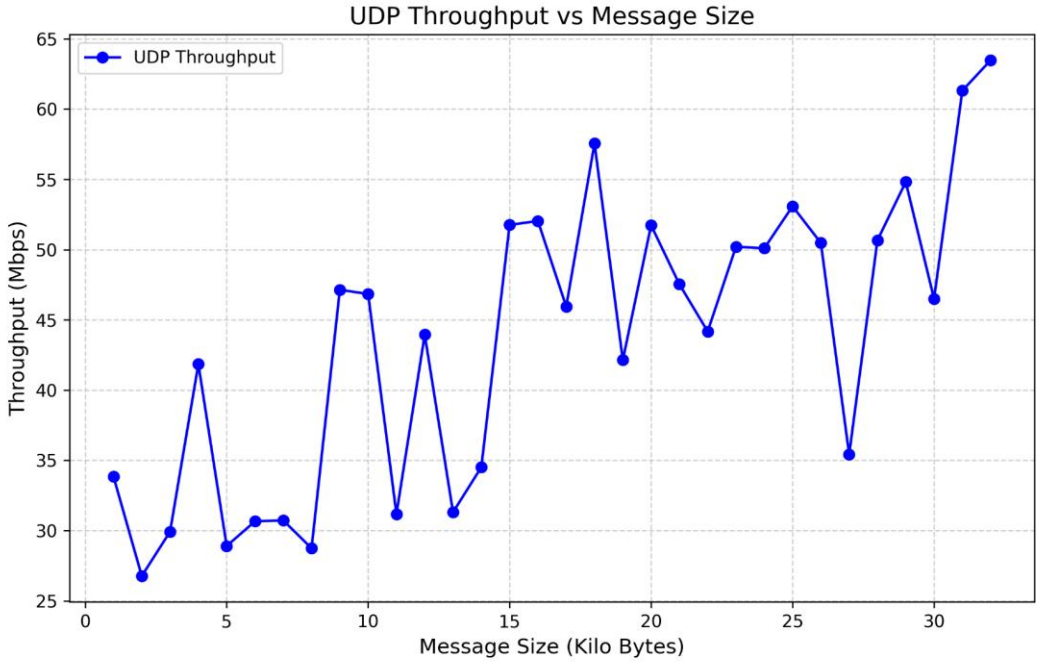
- Throughput peaks and stabilizes:
  - 1 MB (1024 KB) ⟶ ≈ 160 Mbps
  - 2 MB (2048 KB) ⟶ ≈ 220 Mbps
  - 5 MB (5120 KB) ⟶ ≈ 225 Mbps ○ 10

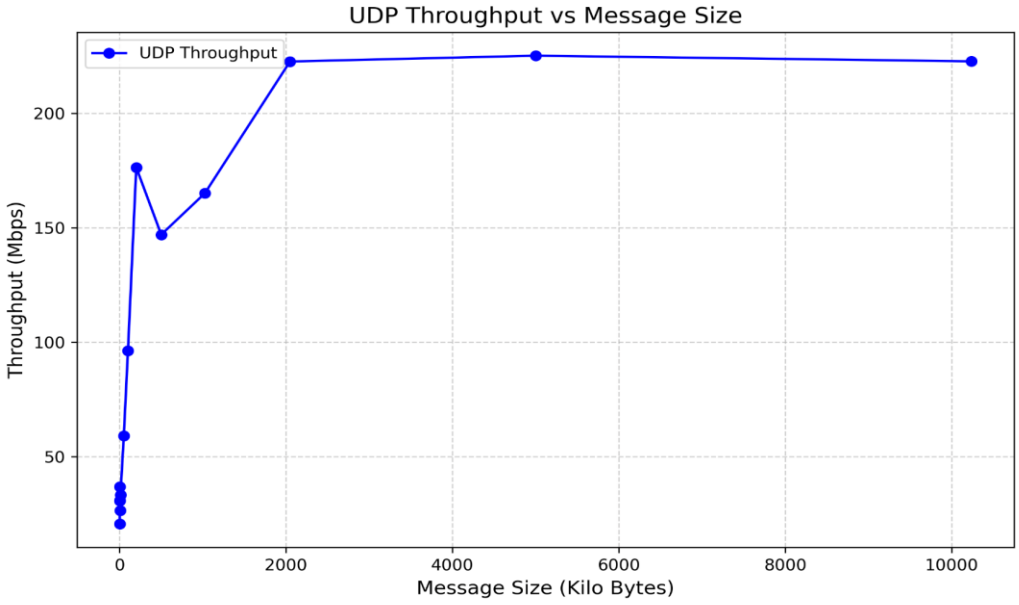
MB (10240 KB) ⟶ ≈ 225 Mbps ● Reason:

- Beyond ~1 MB, UDP throughput saturates.
- The limit is imposed by:
  - Network Interface Card (NIC) maximum capacity.
  - OS UDP buffer limits.
  - Underlying physical bandwidth of the network.

Throughput Trend Table

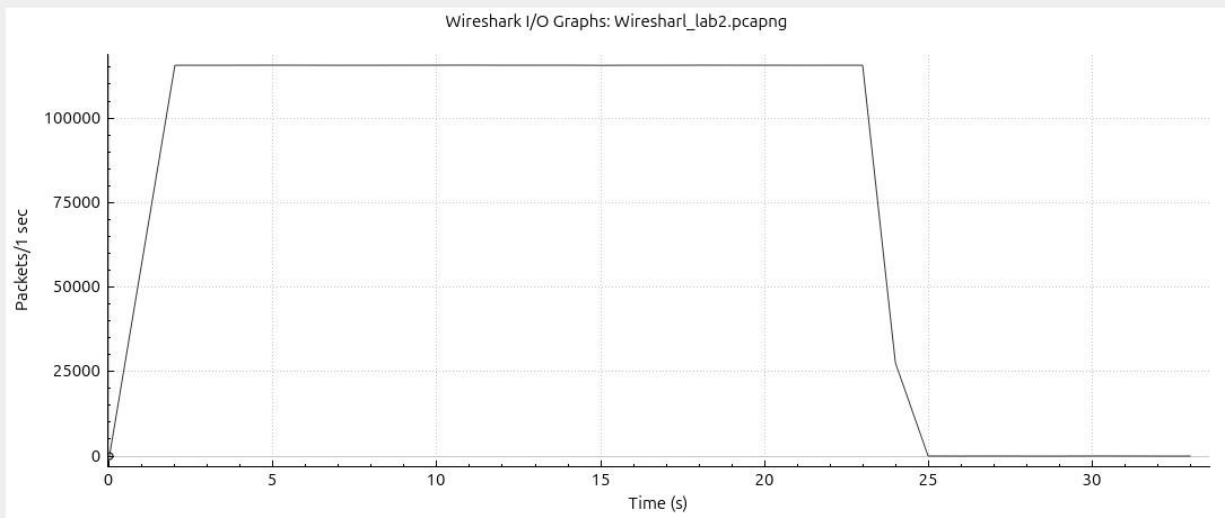
Message Size (KB)	Throughput (Mbps)	Observation
1 – 10 KB	21–36 Mbps	High overhead, unstable
50 – 200 KB	87–108 Mbps	Rapid improvement
500 KB – 1 MB	152–211 Mbps	High efficiency
2 MB – 10 MB	221–224 Mbps	Throughput saturation


UDP Throughput vs Message Size
UDP Throughput
Throughput (Mbps)
Message Size (Kilo Bytes)
65
60
55
50
45
40
35
30
25
0
5
10
15
20
25
30


UDP Throughput vs Message Size
UDP Throughput
Throughput (Mbps)
Message Size (Kilo Bytes)
200
150
100
50
0
2000
4000
6000
8000
10000

Wireshark Analysis

- Sent 10MB data from one pc to another pc in the lab and analysed the packets



18	1.500345533	172.16.114.45	172.16.114.46	UDP	69	40772 → 9386 Len=27
19	1.500345727	172.16.114.45	172.16.114.46	UDP	90	40772 → 9386 Len=48
20	1.500391006	172.16.114.45	172.16.114.46	UDP	66	40772 → 9386 Len=24
21	1.500391167	172.16.114.45	172.16.114.46	UDP	92	40772 → 9386 Len=50
22	1.500391280	172.16.114.45	172.16.114.46	UDP	84	40772 → 9386 Len=42
23	1.500391385	172.16.114.45	172.16.114.46	UDP	91	40772 → 9386 Len=49
24	1.500391495	172.16.114.45	172.16.114.46	UDP	85	40772 → 9386 Len=43
25	1.500431157	172.16.114.45	172.16.114.46	UDP	90	40772 → 9386 Len=48
26	1.500431293	172.16.114.45	172.16.114.46	UDP	88	40772 → 9386 Len=46
27	1.500431402	172.16.114.45	172.16.114.46	UDP	90	40772 → 9386 Len=48
28	1.500431510	172.16.114.45	172.16.114.46	UDP	72	40772 → 9386 Len=30
29	1.500431619	172.16.114.45	172.16.114.46	UDP	92	40772 → 9386 Len=50
30	1.500474428	172.16.114.45	172.16.114.46	UDP	67	40772 → 9386 Len=25
31	1.500474546	172.16.114.45	172.16.114.46	UDP	89	40772 → 9386 Len=47
32	1.500474656	172.16.114.45	172.16.114.46	UDP	87	40772 → 9386 Len=45
33	1.500474762	172.16.114.45	172.16.114.46	UDP	90	40772 → 9386 Len=48
34	1.500513283	172.16.114.45	172.16.114.46	UDP	93	40772 → 9386 Len=51
35	1.500513418	172.16.114.45	172.16.114.46	UDP	89	40772 → 9386 Len=47
36	1.500513526	172.16.114.45	172.16.114.46	UDP	89	40772 → 9386 Len=47
37	1.500513632	172.16.114.45	172.16.114.46	UDP	89	40772 → 9386 Len=47
38	1.500513790	172.16.114.45	172.16.114.46	UDP	91	40772 → 9386 Len=49
39	1.500555028	172.16.114.45	172.16.114.46	UDP	85	40772 → 9386 Len=43
40	1.500555158	172.16.114.45	172.16.114.46	UDP	95	40772 → 9386 Len=53
41	1.500555267	172.16.114.45	172.16.114.46	UDP	84	40772 → 9386 Len=42
42	1.500555376	172.16.114.45	172.16.114.46	UDP	66	40772 → 9386 Len=24
43	1.500555484	172.16.114.45	172.16.114.46	UDP	88	40772 → 9386 Len=46

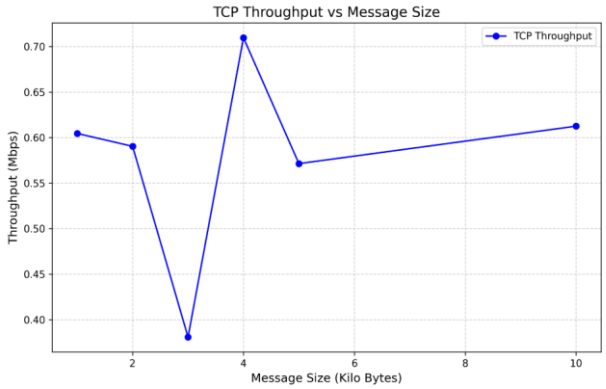
8	1.498358820	172.16.114.45	172.16.114.46	TCP	74	43878 → 8080 [SYN] Seq=0 Win=64240
9	1.498408152	172.16.114.46	172.16.114.45	TCP	74	8080 → 43878 [SYN, ACK] Seq=0 Ack=
10	1.498610439	172.16.114.45	172.16.114.46	TCP	66	43878 → 8080 [ACK] Seq=1 Ack=1 Win=
11	1.498853879	172.16.114.45	172.16.114.46	TCP	91	43878 → 8080 [PSH, ACK] Seq=1 Ack=
12	1.498883151	172.16.114.46	172.16.114.45	TCP	66	8080 → 43878 [ACK] Seq=1 Ack=26 Wi
13	1.498961228	172.16.114.46	172.16.114.45	TCP	74	8080 → 43878 [PSH, ACK] Seq=1 Ack=
14	1.499007672	172.16.114.46	172.16.114.45	TCP	66	8080 → 43878 [FIN, ACK] Seq=9 Ack=
15	1.499116395	172.16.114.45	172.16.114.46	TCP	66	43878 → 8080 [ACK] Seq=26 Ack=9 Wi
16	1.499265996	172.16.114.45	172.16.114.46	TCP	66	43878 → 8080 [FIN, ACK] Seq=26 Ack=

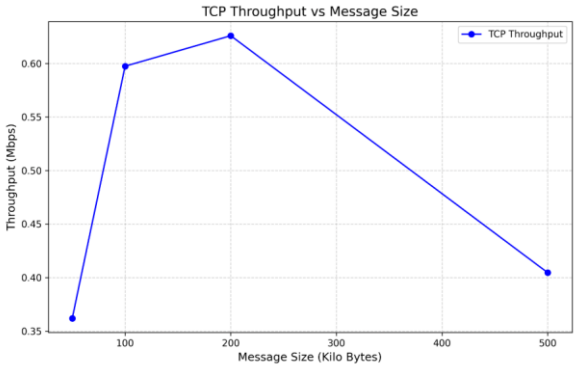
Wireshark · Protocol Hierarchy Statistics · Wiresharl_lab2.pcapng

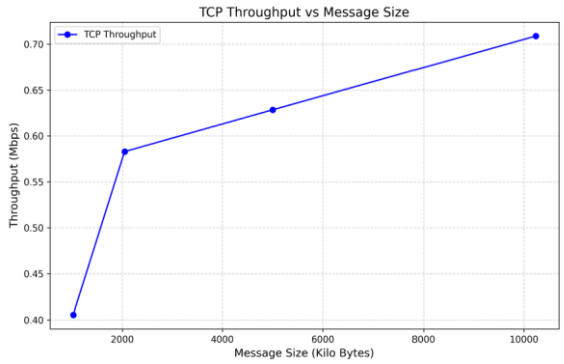
Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	2629189	100.0	221155109	52 M	0	0	0	2629189
Ethernet	100.0	2629189	16.6	36809487	8,767 k	0	0	0	2629189
Media Independent Network Transport	0.0	1	0.0	70	16	1	70	16	1
Logical-Link Control	0.0	7	0.0	563	134	0	0	0	7
HP Extended Local-Link Control	0.0	5	0.0	35	8	0	0	0	5
HP Switch Protocol	0.0	5	0.0	445	105	5	445	105	5
Dynamic Trunk Protocol	0.0	2	0.0	52	12	2	52	12	2
Link Layer Discovery Protocol	0.0	1	0.0	251	59	1	251	59	1
Internet Protocol Version 4	100.0	2629133	23.8	52582660	12 M	0	0	0	2629133
User Datagram Protocol	100.0	2629054	9.5	21032432	5,009 k	0	0	0	2629054
Simple Service Discovery Protocol	0.0	53	0.0	9646	2,297	53	9646	2,297	53
Multicast Domain Name System	0.0	1	0.0	310	73	1	310	73	1
Link-local Multicast Name Resolution	0.0	1	0.0	38	9	1	38	9	1
Dynamic Host Configuration Protocol	0.0	1	0.0	300	71	1	300	71	1
Dropbox LAN sync Discovery Protocol	0.0	5	0.0	785	186	5	785	186	5
Domain Name System	0.0	4	0.0	833	198	4	833	198	4
Data	100.0	2628989	50.1	110707781	26 M	2628989	110707781	26 M	2628989
Transmission Control Protocol	0.0	79	0.0	8610	2,050	55	2948	702	79
Transport Layer Security	0.0	24	0.0	5470	1,302	24	5470	1,302	24
Address Resolution Protocol	0.0	47	0.0	2126	506	47	2126	506	47
Cisco ISL	0.0	1	0.0	26	6	0	0	0	1

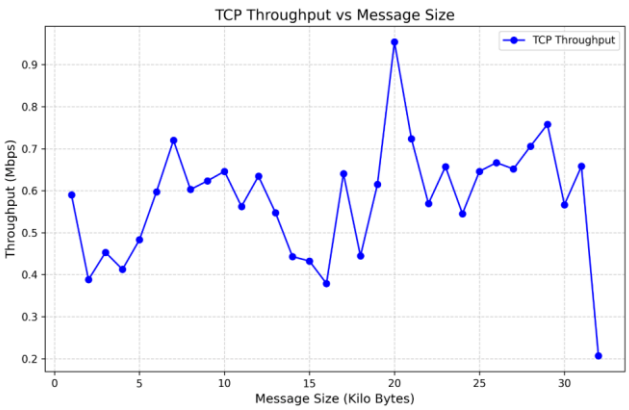
TCP Results

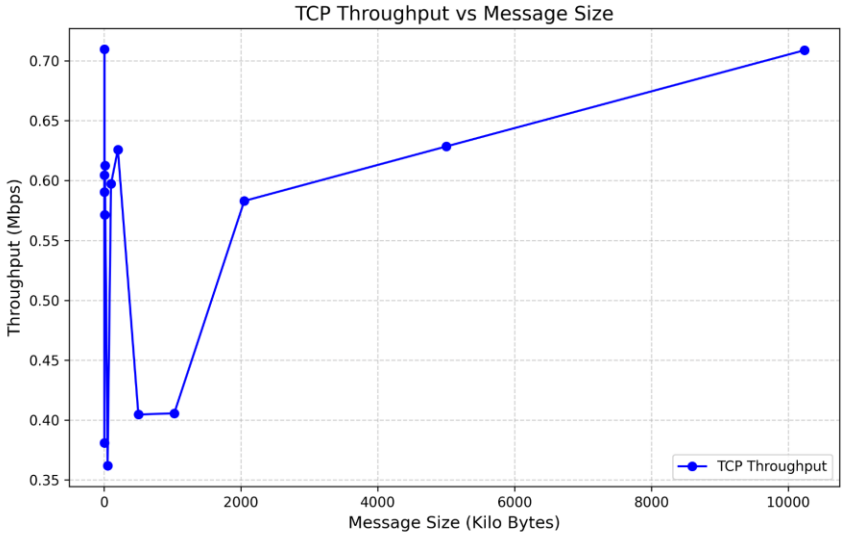
- TCP throughput is lower compared to UDP for this setup (due to handshake, acknowledgments, and reliability features).
- Performance remains fairly constant across message sizes.











Observations

- TCP ensures reliability but is slower for small messages.
- UDP achieves higher throughput as message size increases.
- FCFS scheduling gives predictable order but may starve later clients.
- Round-Robin ensures fairness among clients.
- Performance measurement confirms that message size directly impacts throughput.

Challenges Faced

- **Packet Loss (UDP):** Since UDP does not guarantee delivery, some packets were lost during large file transfers, slightly reducing throughput accuracy.
- **Out-of-Order Packets (UDP):** Packets sometimes arrive out of sequence, requiring extra care while analyzing results.
- **Protocol Overhead (small messages):** For small message sizes (1 KB → 10 KB), TCP/UDP/IP/Ethernet headers created high relative overhead, reducing throughput.
- **Buffer Size Limitations:** OS/network buffer limits caused saturation beyond certain message sizes (~1 MB and above).
- **TCP Connection Setup Delay:** The three-way handshake introduced a small initial latency, affecting performance for very small file transfers.
- **System Call and Context Switching Overhead:** Frequent send/recv calls for small chunks increased CPU overhead.

- **Wireshark Capture Issues:** Capturing long runs created very large trace files, requiring filtering and careful analysis.
- **Iteration Handling:** Repeated file transfers (multiple iterations) sometimes caused timing variations, especially under system load.

Conclusion

This lab helped us understand:

- Practical use of TCP for negotiation and UDP for data transfer.
- Implementation of a multi-threaded concurrent server.
- Impact of scheduling policies on client servicing.
- Measuring and analyzing throughput performance in real networks.